

NeuroSky MindWave Mobile Windows SDK

Developer Guide · v2.0.3

Table of Contents

1. [Overview](#)
 2. [How It Works — Architecture](#)
 3. [Requirements](#)
 4. [Installation](#)
 5. [Windows Setup & Deployment](#)
 6. [Quick Start](#)
 7. [Finding Your Device MAC Address](#)
 8. [Connection Modes & State Machine](#)
 9. [EEG Data Model](#)
 10. [EEG Frequency Bands Explained](#)
 11. [Signal Quality](#)
 12. [Commands](#)
 13. [Simulator — Develop Without Hardware](#)
 14. [Error Handling & Reconnection](#)
 15. [Advanced Patterns](#)
 16. [Troubleshooting](#)
 17. [Testing](#)
 18. [API Reference](#)
-

1. Overview

The **NeuroSky MindWave Mobile Windows SDK** is a modern C# library that lets you read real-time EEG (electroencephalography) data from a NeuroSky MindWave Mobile headset on Windows 10 or later — with zero dependency on NeuroSky's legacy ThinkGear Connector (TGC) software.

Why this SDK exists

The official NeuroSky SDK requires TGC, a background Windows service, to be running on the user's machine. This creates friction: users must install and start a separate process, troubleshoot port conflicts, and deal with a heavyweight dependency that is difficult to bundle in modern applications.

This SDK eliminates TGC entirely by communicating directly with the MindWave Mobile hardware via the Windows Bluetooth stack (WinRT). Your application talks to the headset directly — no intermediary service, no installer prerequisite beyond .NET 8.

Key features

Feature	Description
No TGC dependency	Communicates with hardware directly via WinRT
BLE + BT Classic	BLE by default; BT Classic available for noisy RF environments

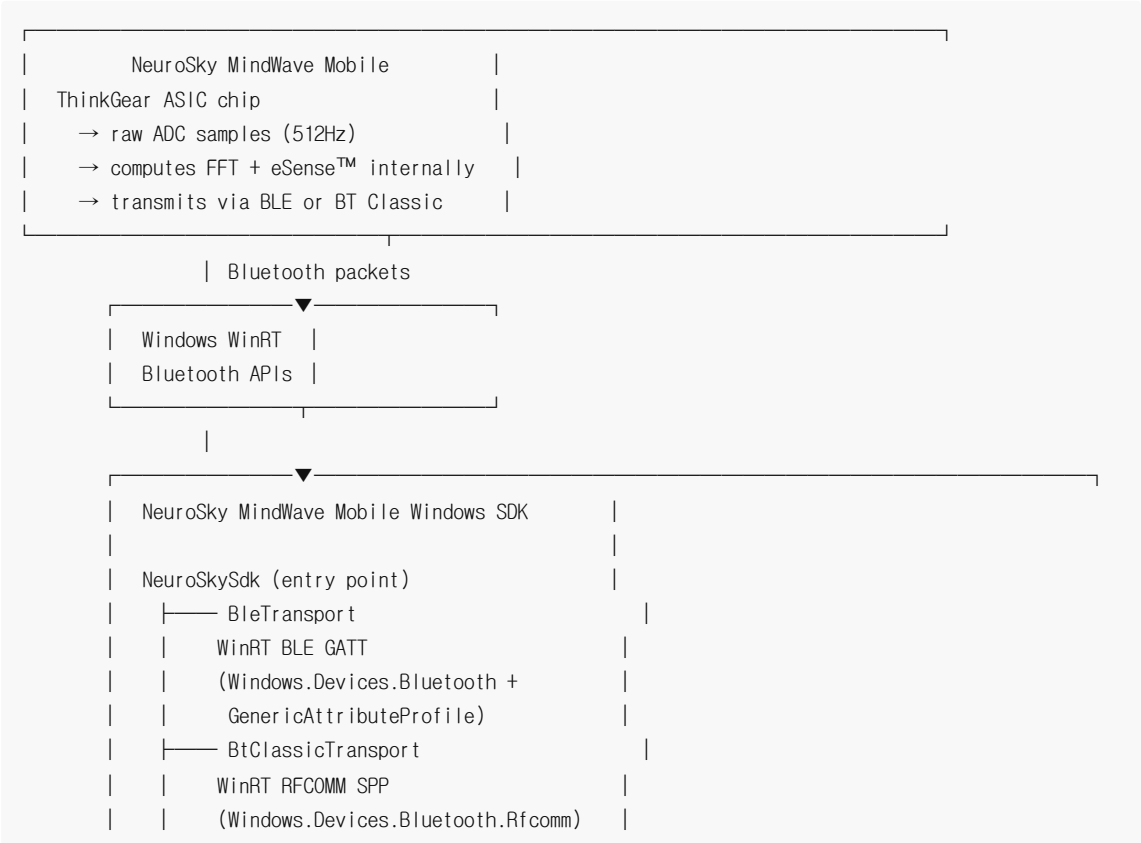
Developer-selectable transport	TransportMode.Ble (default) or TransportMode.BtClassic — no hidden auto-fallback
Async stream API	IEnumerable<BrainWaveData> — native await foreach, cancel via CancellationToken
Built-in Simulator	Full data simulation without any hardware
Trimmer / AOT safe	Ships an internal TrimmerRootDescriptor — no consumer setup required
.NET 8 / C# 12	Modern language features, nullable annotations, file-scoped namespaces
NuGet distribution	One-line package reference: NeuroSky.MindWave.Sdk

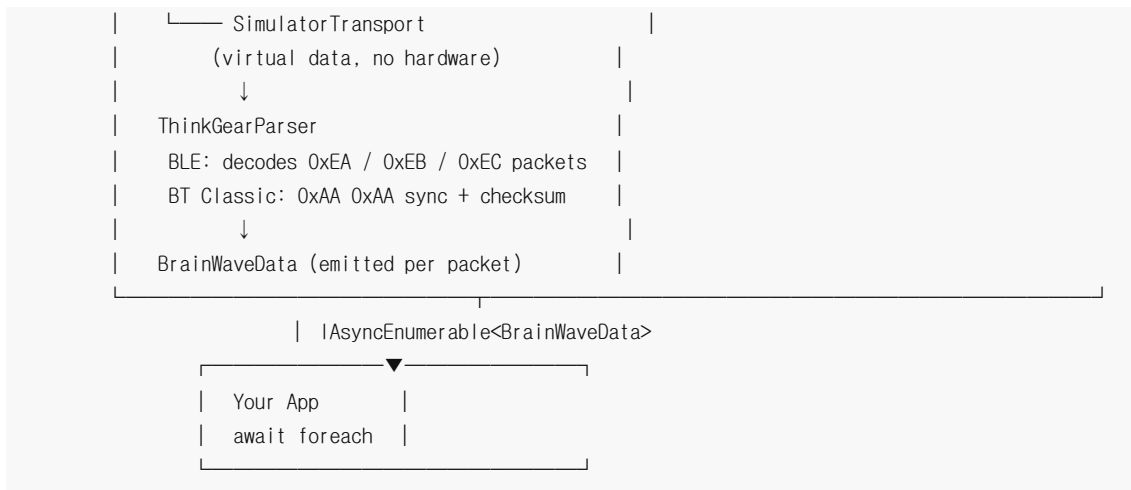
What you can measure

The MindWave Mobile headset contains a single dry electrode on the forehead (FP1 position) and a reference clip on the ear. From this single channel, the ThinkGear ASIC chip on board computes:

- **Raw EEG waveform** — 512 samples/sec, signed 16-bit values
- **8 frequency band powers** — Delta, Theta, Alpha (Low/High), Beta (Low/High), Gamma (Low/Mid)
- **eSense™ Attention** — NeuroSky's proprietary attention index (0~100)
- **eSense™ Meditation** — NeuroSky's proprietary relaxation index (0~100)
- **Eye blink detection** — intensity 0~255
- **Signal quality** — 0 (perfect contact) to 200 (no signal)

2. How It Works — Architecture





BLE vs BT Classic — internal differences

BLE (Bluetooth Low Energy) path: The MindWave Mobile exposes three BLE GATT characteristics:

- `039afff8-...` — eSense data (Attention, Meditation, frequency bands) — SDK subscribes to notifications
- `039afff4-...` — Raw EEG data — SDK subscribes to notifications
- `039affa0-...` — Handshake characteristic — SDK writes command bytes to start data flow

BT Classic (RFCOMM SPP) path: The MindWave Mobile emulates a serial port (Serial Port Profile, UUID `00001101-...`) at 57 600 baud. The SDK opens an RFCOMM socket and reads a continuous byte stream. `ThinkGearParser` synchronizes on the `0xAA 0xAA` sync bytes and parses variable-length payload codes (0x02 PoorSignal, 0x04 Attention, 0x05 Meditation, 0x80 Raw EEG, 0x83 EEG Power).

Both paths produce identical `BrainWaveData` output. The parsing layer is shared.

BLE default — no automatic fallback

`NeuroSkySdk.ConnectAsync()` uses BLE by default. To use BT Classic instead, pass `TransportMode.BtClassic` to `ConnectAsync()`. Both transports produce the same `DataStream` output.

*NeuroSkySdk does **not** attempt BLE first and then fall back to BT Classic automatically. The transport you pass to `ConnectAsync()` is the only transport used. If you need fallback logic, implement it yourself in the caller.*

Single flat namespace

As of v2.0.1, every SDK type lives in the single `NeuroSky.Sdk` namespace. A single `using NeuroSky.Sdk;` is sufficient — there are no sub-namespaces like `Transport`, `Parser`, or `Model` to import.

3. Requirements

System requirements

Component	Minimum	Notes
Windows	Windows 10 version 1903 (build 18362)	WinRT BLE GATT requires 1903+

.NET runtime	.NET 8.0	Must be installed on the target machine
Bluetooth adapter	BLE-capable adapter	For TransportMode.Ble (default)
Bluetooth adapter	Classic BT adapter	For TransportMode.BtClassic
Device pairing	Not required for BLE	Required for BT Classic

Project requirements

Your application's `.csproj` must target the Windows platform TFM (Target Framework Moniker) to access WinRT APIs:

```
<!-- Required - standard net8.0 will NOT work -->
<TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
```

The `10.0.19041.0` suffix corresponds to Windows 10 version 2004. This is the minimum build required for stable WinRT BLE GATT support. Targeting this version does **not** prevent the app from running on newer Windows 10/11 builds.

Important: If your project omits the `-windows10.0.19041.0` suffix and targets plain `net8.0`, the WinRT types (`Windows.Devices.*`) will not be available and the SDK will throw `PlatformNotSupportedException` at runtime.

Supported headset

This SDK is designed and tested for the **NeuroSky MindWave Mobile 2** (sometimes labeled just "MindWave Mobile"). The original MindWave (wired, USB dongle) is not supported. Both BLE and BT Classic modes of the MindWave Mobile 2 are supported.

4. Installation

Option A — NuGet Package Manager UI (Visual Studio)

1. Right-click your project in Solution Explorer → **Manage NuGet Packages**
2. Select the **Browse** tab
3. Search for: `NeuroSky.MindWave.Sdk`
4. Click **Install**

Option B — Edit `.csproj` directly (recommended for CI/CD)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <!-- Windows TFM required for WinRT APIs -->
    <TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="NeuroSky.MindWave.Sdk" Version="2.0.3" />
  </ItemGroup>
</Project>
```

```
</ItemGroup>
</Project>
```

Option C — .NET CLI

```
dotnet add package NeuroSky.MindWave.Sdk --version 2.0.3
```

Verify installation

After installing, confirm the package resolves correctly:

```
dotnet restore
dotnet build
```

If you see `CS0246: The type or namespace name 'NeuroSkySdk' could not be found`, check that:

1. The package is listed in `.csproj`
2. `TargetFramework` includes the `-windows` suffix
3. You have added `using NeuroSky.Sdk;` at the top of your file

5. Windows Setup & Deployment

This section covers everything specific to running the SDK on Windows beyond the basic NuGet reference: enabling Bluetooth, packaging considerations, and how to publish trimmed / self-contained / AOT binaries safely.

Bluetooth permissions

Unlike Android, Windows desktop (`net8.0-windows...`) apps **do not** require runtime permission requests for Bluetooth. The SDK uses `Windows.Devices.Bluetooth.*` APIs directly and the OS gates access at the device level rather than per-app.

For **MSIX-packaged apps** (Windows Store, packaged WPF/WinUI), you must declare the Bluetooth capability in `Package.appxmanifest` :

```
<Capabilities>
  <DeviceCapability Name="bluetooth" />
</Capabilities>
```

Unpackaged console / WPF / WinForms applications do not need this declaration.

Trimming / Self-contained / AOT

The package is marked `IsTrimmable=true` and ships an internal `TrimmerRootDescriptor.xml` that roots the transport, parser, and public API types. **No consumer action is required** — publishing trimmed, self-contained, or with `PublishAot=true` works out of the box.

```
<PropertyGroup>
  <PublishTrimmed>true</PublishTrimmed>
  <SelfContained>true</SelfContained>
```

```
<RuntimeIdentifier>win-x64</RuntimeIdentifier>
</PropertyGroup>
```

```
dotnet publish -c Release
```

Why this matters: WinRT GATT notification handlers are dispatched by the Windows Bluetooth stack through reflection-like mechanisms. Without the shipped descriptor, the .NET trimmer would silently remove the handler methods in trimmed/AOT builds — the BLE link would establish, but `DataStream` would emit nothing. v2.0.3 corrects a descriptor namespace mismatch that previously made this protection ineffective; do not pin to v2.0.1 or v2.0.2 if you publish trimmed.

Distributing your application

Distribution	TFM impact	Notes
Self-contained (.exe + runtime)	Same TFM	Largest output; no .NET install required on target
Framework-dependent	Same TFM	Smallest output; .NET 8.0 Desktop Runtime must be pre-installed
MSIX packaged	Same TFM + capability	Required for Store; needs <code><DeviceCapability Name="bluetooth"/></code>
<code>PublishAot=true</code>	Same TFM	Native binary; uses the shipped trimmer descriptor automatically

6. Quick Start

The following example is a complete minimal application that connects to a MindWave Mobile headset and streams EEG data until the user presses Ctrl+C.

```
using NeuroSky.Sdk;

// Step 1: Create the SDK instance.
// NeuroSkySdk is IDisposable - use 'await using' so it disconnects
// cleanly when the block exits (including on exception or Ctrl+C).
await using var sdk = new NeuroSkySdk();

// Step 2: Subscribe to connection state changes (optional but recommended).
// ConnectAsync NEVER throws on connection failure - it transitions to
// ConnectionState.Error instead, so observe StateChanged to catch failures.
sdk.StateChanged += (_, state) =>
{
    Console.WriteLine($"[State] {state}");
    if (state == ConnectionState.Error)
        Console.WriteLine("Connection failed - verify pairing / power / MAC.");
};

// Step 3: Set up graceful cancellation.
```

```

using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) =>
{
    e.Cancel = true; // prevent immediate process termination
    cts.Cancel();    // signal the data stream to stop
};

// Step 4: Connect to the headset.
// Replace with your MindWave Mobile's actual MAC address.
// Default mode is BLE. Pass TransportMode.BtClassic for BT Classic.
// See Section 7 for how to find your MAC address.
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");

// Step 5: Set the notch filter for your region (recommended).
// This removes 50Hz or 60Hz power-line noise from raw EEG.
await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz); // Korea / USA
// await sdk.SendCommandAsync(NeuroSkyCommand.Notch50Hz); // Europe / China

// Step 6: Stream data.
// DataStream() returns IEnumerable<BrainWaveData>.
// Each iteration yields one parsed EEG packet.
await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.SignalQuality == SignalQuality.NoSignal)
    {
        Console.WriteLine("No signal - adjust the headset.");
        continue;
    }

    Console.WriteLine($"Attention : {data.Attention,3} " +
        $"Meditation: {data.Meditation,3} " +
        $"Signal      : {data.SignalQuality}");
}

```

Connecting by MAC address

`ConnectAsync` takes a Bluetooth MAC address in `AA:BB:CC:DD:EE:FF` format. Unlike the Android SDK, the Windows SDK does **not** accept a device name in `ConnectAsync` — use `FindDeviceAddressAsync()` once on first launch to resolve the name to a MAC, then cache that MAC for subsequent runs.

See [Section 7](#) for the complete discovery flow.

7. Finding Your Device MAC Address

Before calling `ConnectAsync`, you need the Bluetooth MAC address of your MindWave Mobile headset. The MAC address is a 12-character hexadecimal identifier in the format `AA:BB:CC:DD:EE:FF`.

Method 1 — `FindDeviceAddressAsync()` (recommended for applications)

The SDK provides a built-in BLE advertisement scan that resolves a device name to a MAC address. Call this once on first launch, cache the result, and skip the scan on subsequent launches.

```

await using var sdk = new NeuroSkySdk();

// Try cache first; fall back to BLE scan (up to 10 s)
var cached = Properties.Settings.Default.DeviceMac;
var address = !string.IsNullOrEmpty(cached)
    ? cached
    : await sdk.FindDeviceAddressAsync("MindWave Mobile");

if (address is null)
{
    Console.WriteLine("Device not found within timeout – check power and BLE adapter.");
    return;
}

// Cache for next launch – avoids the scan delay
Properties.Settings.Default.DeviceMac = address;
Properties.Settings.Default.Save();

await sdk.ConnectAsync(address);

```

Signature:

```

Task<string?> FindDeviceAddressAsync(
    string deviceName,
    int timeoutMs = 10_000,
    CancellationToken ct = default)

```

Parameter	Default	Description
deviceName	—	BLE advertisement name to match (exact, case-sensitive)
timeoutMs	10000	How long to scan before returning null
ct	—	Cancellation token; cancels the scan immediately

Returns the MAC address as "AA:BB:CC:DD:EE:FF", or null if not found within the timeout.

Method 2 — Windows Settings (easiest for one-off lookup)

1. Turn on the MindWave Mobile headset (power switch on the left side)
2. Open **Settings** → **Bluetooth & other devices**
3. Pair the device if not already paired (no PIN required)
4. Click on **"MindWave Mobile" → Properties** (or **More info**)
5. The MAC address is shown as a 12-digit hex string

Method 3 — PowerShell (for already-paired devices)

```

Get-PnpDevice -Class Bluetooth |
    Where-Object { $_.FriendlyName -like "*MindWave*" } |
    Select-Object FriendlyName, DeviceID

```


Sample output:

FriendlyName	DeviceID
-----	-----
MindWave Mobile	BTHENUMW...W7&3A1B2C3D&0&AABBCCDDEEFF_C00000000

The last 12 hex characters before `_C00000000` are your MAC address. Format them as `AA:BB:CC:DD:EE:FF`.

Method 4 — Bluetooth LE Explorer app (visual scan)

If the device is not yet paired and you want to scan for its MAC address without pairing:

1. Install **Bluetooth LE Explorer** from the Microsoft Store (free, official Microsoft tool)
2. Open the app and click **Start**
3. Turn on your MindWave Mobile headset
4. Look for a device named "MindWave Mobile" in the scan results
5. The address shown is your MAC address

8. Connection Modes & State Machine

The `TransportMode` enum

```
public enum TransportMode
{
    Ble,           // BLE – no Windows pairing required (default)
    BtClassic      // BT Classic – requires Windows Bluetooth pairing
}
```

`TransportMode.Ble` (default)

```
// Both lines are equivalent:
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Ble);
```

How it works: WinRT's `BluetoothLEDevice.FromBluetoothAddressAsync()` opens a GATT session to the MindWave Mobile, the SDK discovers services, subscribes to eSense and Raw EEG characteristics, and writes the `0x17` (StartESense) handshake command to begin data streaming.

When to use:

- Your users should not need to manually pair the device
- The target machine has a BLE-capable adapter (most adapters made after 2012)
- You want the smoothest user experience

`TransportMode.BtClassic`

```
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.BtClassic);
```

How it works: WinRT's `RfcommDeviceService` opens an RFCOMM socket over Serial Port Profile. The MindWave Mobile presents itself as a virtual serial port at 57 600 baud. The `ThinkGearParser` reads the incoming byte

stream and synchronizes on `0xAA 0xAA` sync headers.

Pairing prerequisite (one-time, per machine):

- 1. Open **Settings** → **Bluetooth & other devices** → **Add device**
- 2. Select **Bluetooth**
- 3. Wait for **"MindWave Mobile"** to appear
- 4. Click it and follow the pairing prompt (no PIN required)
- 5. Confirm the device shows as **Paired**

When to use:

- You are deploying to a controlled environment where devices are pre-paired by IT
- BLE connectivity is unreliable on the target hardware
- You are integrating with existing BT Classic infrastructure

`ConnectionState` **machine**

Every transport progresses through the same lifecycle states:



State	Meaning
Disconnected	Initial state, or after <code>DisconnectAsync()</code> / link drop
Scanning	BLE only — resolving the MAC address
Connecting	GATT discovery (BLE) or RFCOMM socket open (BT Classic) in progress
Connected	Notifications enabled and handshake sent — <code>DataStream</code> will emit packets
Error	Device not found, GATT discovery failed, handshake characteristic missing , or RFCOMM service unavailable. <code>DataStream</code> will not emit; call <code>DisconnectAsync()</code> and retry.

`ConnectAsync` **never throws on connection failure**. Instead it transitions to `ConnectionState.Error`. Subscribing to `StateChanged` (or polling `sdk.State`) is the **only** way to detect connection failures. Code that wraps `ConnectAsync` in `try/catch` alone will miss BLE failures.

```
sdk.StateChanged += (_, state) =>
{
```

```

switch (state)
{
    case ConnectionState.Scanning:
        statusLabel.Text = "Searching for MindWave Mobile...";
        break;
    case ConnectionState.Connecting:
        statusLabel.Text = "Connecting...";
        break;
    case ConnectionState.Connected:
        statusLabel.Text = "Connected";
        connectButton.IsEnabled = false;
        disconnectButton.IsEnabled = true;
        break;
    case ConnectionState.Error:
        statusLabel.Text = "Connection failed. Check Bluetooth and try again.";
        break;
    case ConnectionState.Disconnected:
        statusLabel.Text = "Disconnected";
        connectButton.IsEnabled = true;
        disconnectButton.IsEnabled = false;
        break;
}
};

```

9. EEG Data Model

`DataStream()` yields a `BrainWaveData` record for each packet received from the MindWave Mobile headset.

```

public record BrainWaveData
{
    public long Timestamp { get; init; } // Unix ms (UTC) at receipt
    public int PoorSignal { get; init; } // 0 = perfect, 200 = no signal
    public int Attention { get; init; } // 0~100, eSense™ attention
    public int Meditation { get; init; } // 0~100, eSense™ meditation
    public int Delta { get; init; } // 0.5~2.75 Hz
    public int Theta { get; init; } // 3.5~6.75 Hz
    public int LowAlpha { get; init; } // 7.5~9.25 Hz
    public int HighAlpha { get; init; } // 10~11.75 Hz
    public int LowBeta { get; init; } // 13~16.75 Hz
    public int HighBeta { get; init; } // 18~29.75 Hz
    public int LowGamma { get; init; } // 31~39.75 Hz
    public int MidGamma { get; init; } // 41~49.75 Hz
    public IReadOnlyList<int> RawEeg { get; init; } // 10 samples/packet, 512Hz
    public int EyeBlink { get; init; } // 0 = no blink, 1~255 = intensity

    public SignalQuality SignalQuality { get; } // derived from PoorSignal
}

```

Data update rates

Field(s)	Update rate	Notes
PoorSignal	~1 Hz	Updated every eSense packet
Attention, Meditation	~1 Hz	eSense™ computed once per second
Delta through MidGamma	~1 Hz	FFT computed once per second
RawEeg	512 Hz total	10 samples per BLE notify, ~51 packets/sec
EyeBlink	Event-driven	Only non-zero when a blink is detected

Important: When `RawEeg` packets arrive, `Attention`, `Meditation`, and `frequency band` fields will be 0 in that `BrainWaveData` object — they are only populated in the eSense packet which arrives separately. The parser **does** accumulate state across packets in BLE mode, so the most recently seen value remains in subsequent emits — but a fresh `RawEeg`-only emit will not refresh the eSense fields. Filter by checking which fields are non-zero, or handle each packet type independently.

Working with timestamps

`Timestamp` is the SDK-side receive time in Unix milliseconds (UTC):

```
var receivedAt = DateTimeOffset.FromUnixTimeMilliseconds(data.Timestamp);
Console.WriteLine($"Packet received at: {receivedAt:HH:mm:ss.fff}");
```

To align EEG samples with external events (e.g., stimulus timing in a BCI experiment), record the `Timestamp` alongside each data point and synchronize using a shared clock reference.

10. EEG Frequency Bands Explained

The MindWave Mobile's ThinkGear chip performs a Fast Fourier Transform (FFT) on the raw EEG and outputs the power in 8 frequency bands.

Important: values are relative, not absolute

The frequency band values are **relative power** in arbitrary units — they are not calibrated to physical units ($\mu\text{V}^2/\text{Hz}$). This means:

- You **cannot** compare values across different sessions or individuals in absolute terms
- You **can** compare values within a single session — e.g., "Delta rose 40 % after eyes closed"
- Ratios** between bands are more meaningful than raw values — e.g., `theta / (alpha + beta)` for attention estimation

Band reference table

Property	Greek	Range	Hz	Typical mental states
Delta	δ	Slow	0.5~2.75 Hz	Deep sleep, healing. High delta while awake = fatigue or poor signal.
Theta	θ	Slow	3.5~6.75 Hz	Drowsiness, daydreaming, creativity, REM sleep, deep meditation.

LowAlpha	α low	Medium	7.5~9.25 Hz	Relaxed, unfocused, calm. Increases with eyes closed.
HighAlpha	α high	Medium	10~11.75 Hz	Eyes-closed rest. Suppressed by active visual attention.
LowBeta	β low	Fast	13~16.75 Hz	Active focus, alert thinking. The "work" band.
HighBeta	β high	Fast	18~29.75 Hz	High arousal, stress, anxiety, intense cognition.
LowGamma	γ low	Very fast	31~39.75 Hz	Higher cognition, cross-modal perception, binding.
MidGamma	γ mid	Very fast	41~49.75 Hz	Intense concentration. Elevated in expert meditators.

eSense™ Attention and Meditation

These are NeuroSky's **proprietary processed values** computed by the ThinkGear chip itself. The SDK receives them pre-computed.

Attention (0~100) — mental focus or concentration:

Range	Meaning
0	Not yet computed (startup, or no signal)
1~40	Low — distracted, relaxed, wandering mind
40~60	Neutral baseline
60~80	Moderate focus — engaged in task
80~100	High focus — strong active concentration

Meditation (0~100) — mental calmness or relaxation:

Range	Meaning
0	Not yet computed (startup, or no signal)
1~40	Low — active thinking, stress
40~60	Neutral baseline
60~80	Moderate relaxation
80~100	Deep calm — strong meditative state

eSense values require 10~20 seconds to stabilize after the headset is put on. Values of 0 at startup are normal.

Raw EEG

When Raw EEG is enabled (via `StartRawEeg` command), `RawEeg` contains 10 signed 16-bit ADC samples per packet at 512 Hz:

```
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

await foreach (var data in sdk.DataStream(cts.Token))
{
    foreach (var sample in data.RawEeg)
    {
        // Each sample: -32768 to +32767
        // 512 Hz, 10 samples per packet ≈ 51 packets/sec
        PlotSample(sample);
    }
}
```

Raw EEG is useful for custom FFT analysis, artifact detection, or research. It is **disabled by default** to reduce Bluetooth bandwidth.

11. Signal Quality

Signal quality is the most critical factor for usable data. Always check it before using attention, meditation, or band values.

PoorSignal values

Value	SignalQuality	Reliability	Action
0	Good	Excellent	Use all data freely
1~50	Fair	Acceptable	Minor noise, eSense still valid
51~199	Poor	Unreliable	Prompt user to adjust headset
200	NoSignal	No data	Headset not worn

Recommended check pattern

```
await foreach (var data in sdk.DataStream(cts.Token))
{
    switch (data.SignalQuality)
    {
        case SignalQuality.NoSignal:
            ShowMessage("Please put on the MindWave Mobile headset.");
            continue;

        case SignalQuality.Poor:
            ShowMessage($"Weak signal ({data.PoorSignal}). Adjust the headset.");
            continue;

        case SignalQuality.Fair:
        case SignalQuality.Good:
```

```

        UpdateAttentionUi(data.Attention);
        UpdateMeditationUi(data.Meditation);
        break;
    }
}

```

Tips for improving signal quality

1. **Moisten the sensor** — a small drop of water on the forehead sensor significantly improves conductance
2. **Clean the forehead** — remove sunscreen, makeup, or sweat residue
3. **Adjust headset position** — center the sensor on FP1 (above the left eyebrow)
4. **Check the ear clip** — must make firm contact with the earlobe
5. **Wait 20~30 seconds** — after putting on the headset for signal to stabilize
6. **Avoid strong muscle movement** — clenching the jaw creates EMG artifact that looks like a signal-quality drop

12. Commands

After connecting, send control commands to configure the MindWave Mobile headset's behavior.

Notch filter

EEG signals are extremely low amplitude and easily contaminated by AC power-line noise. Send the notch filter command immediately after connecting:

Region	Grid frequency	Command
Korea	60 Hz	NeuroSkyCommand.Notch60Hz
USA / Canada	60 Hz	NeuroSkyCommand.Notch60Hz
Europe	50 Hz	NeuroSkyCommand.Notch50Hz
China	50 Hz	NeuroSkyCommand.Notch50Hz
Australia / UK	50 Hz	NeuroSkyCommand.Notch50Hz

```

await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz); // Korea/USA
// await sdk.SendCommandAsync(NeuroSkyCommand.Notch50Hz); // Europe/China

```

Without the notch filter, you will likely see a large 50 Hz or 60 Hz artifact in raw EEG and elevated Beta band values.

Raw EEG streaming

```

// Enable raw EEG (disabled by default)
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

```

```
// Disable when no longer needed
await sdk.SendCommandAsync(NeuroSkyCommand.StopRawEeg);
```

eSense control

```
// Disable eSense if only raw EEG is needed
await sdk.SendCommandAsync(NeuroSkyCommand.StopESense);

// Re-enable
await sdk.SendCommandAsync(NeuroSkyCommand.StartESense);
```

All commands

Constant	Byte	Description
NeuroSkyCommand.Notch60Hz	0x1C	Notch filter at 60 Hz
NeuroSkyCommand.Notch50Hz	0x1B	Notch filter at 50 Hz
NeuroSkyCommand.StartRawEeg	0x15	Enable raw EEG stream
NeuroSkyCommand.StopRawEeg	0x16	Disable raw EEG stream
NeuroSkyCommand.StartESense	0x17	Enable eSense output (sent automatically on BLE connect)
NeuroSkyCommand.StopESense	0x18	Disable eSense output

The SDK automatically writes `StartESense ( 0x17 )` to the BLE handshake characteristic as the final step of `ConnectAsync( _, TransportMode.Ble )`. You typically only need `Notch60Hz` / `Notch50Hz` and optionally `StartRawEeg`.

13. Simulator — Develop Without Hardware

`SimulatorTransport` generates synthetic EEG data without any MindWave Mobile hardware. It implements the same `ITransport` interface as the real transports, so your application code remains unchanged between development and production.

Why use the Simulator

- **No hardware required** — develop and test UI, data pipelines, and business logic before the headset arrives
- **Predictable data** — use `Focused` mode to always produce high-attention data for UI testing
- **Edge case testing** — `PoorSignal` mode tests your error-handling and reconnect logic
- **CI/CD pipelines** — run automated tests on build servers without Bluetooth hardware

Basic usage

```
using NeuroSky.Sdk;

var simulator = new SimulatorTransport();
simulator.SetMode(SimulatorTransport.Mode.Focused);
```



```

await simulator.ConnectAsync("simulator"); // any string accepted; ~500 ms

await foreach (var data in simulator.DataStream(cts.Token))
{
    Console.WriteLine($"[SIM] Attention: {data.Attention}, " +
        $"Meditation: {data.Meditation}");
}

```

Simulator modes

Mode	Attention	Meditation	PoorSignal	Use case
Random	0~100 (random)	0~100 (random)	0~30	General integration testing
Focused	70~100	40~60	0	High-attention UI testing
Relaxed	20~50	70~100	0	High-meditation UI testing
PoorSignal	0	0	150~200	Signal loss and error handling

Switching modes at runtime

```

var simulator = new SimulatorTransport();
simulator.SetMode(SimulatorTransport.Mode.PoorSignal);
await simulator.ConnectAsync("simulator");

await Task.Delay(5000);
simulator.SetMode(SimulatorTransport.Mode.Focused);

```

Dependency injection — swap simulator and real SDK

Both `NeuroSkySdk` and `SimulatorTransport` implement `ITransport`, so application code can be transport-agnostic. Note that `NeuroSkySdk` itself implements `ITransport` indirectly through its public surface, but for the cleanest DI pattern inject `ITransport` and dispatch in your composition root:

```

ITransport transport;

if (args.Contains("--simulator"))
{
    var sim = new SimulatorTransport();
    sim.SetMode(SimulatorTransport.Mode.Focused);
    await sim.ConnectAsync("simulator");
    transport = sim;
}
else
{
    var sdk = new NeuroSkySdk();
    await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
    // Adapter pattern: wrap NeuroSkySdk to expose ITransport directly,
    // or call sdk.DataStream() through your own facade.
}

```

```

    transport = new SdkTransportAdapter(sdk);
}

await foreach (var data in transport.DataStream(cts.Token))
    ProcessData(data);

```

14. Error Handling & Reconnection

EEG applications often run for extended periods. Robust error handling and automatic reconnection are essential for production use.

Detecting connection failures

`ConnectAsync` does **not** throw on most connection failures — it transitions to `ConnectionState.Error`. Always monitor `StateChanged` (or check `sdk.State` after the call returns):

```

sdk.StateChanged += (_, state) =>
{
    if (state == ConnectionState.Error)
        Console.WriteLine("Connection failed - check Bluetooth and MAC address.");
};

await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");

if (sdk.State == ConnectionState.Error)
{
    // Connection failed - do not start the DataStream loop
    return;
}

```

Conditions that produce `Error` :

- Device not found at the given MAC address
- GATT service discovery failed
- **Handshake characteristic not present** on the device (v2.0.3+ — previously silent)
- RFCOMM service unavailable for the BT Classic transport
- Bluetooth adapter disabled or unavailable

Stream disconnection and auto-reconnect

`DataStream()` ends (the `await foreach` loop exits) when the connection drops. Wrap in a retry loop for production robustness:

```

const string address = "AA:BB:CC:DD:EE:FF";
const int retryDelayMs = 3000;

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        Console.WriteLine("Connecting to MindWave Mobile...");
    }
}

```

```

        await sdk.ConnectAsync(address, TransportMode.Ble, cts.Token);

        if (sdk.State == ConnectionState.Error)
        {
            Console.WriteLine($"Connect failed. Retrying in {retryDelayMs / 1000}s...");
        }
        else
        {
            Console.WriteLine("Connected. Streaming data...");
            await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);

            await foreach (var data in sdk.DataStream(cts.Token))
                ProcessData(data);

            Console.WriteLine("Stream ended. Device may have been turned off.");
        }
    }
    catch (OperationCanceledException) when (cts.Token.IsCancellationRequested)
    {
        Console.WriteLine("Stopped by user.");
        break;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }

    try { await Task.Delay(retryDelayMs, cts.Token); }
    catch (OperationCanceledException) { break; }
}

```

Handling NoSignal without disconnecting

The Bluetooth link can remain active while the electrode is not touching the skin. In this case, `SignalQuality` becomes `NoSignal` but `DataStream` keeps emitting. Handle this in your collector:

```

await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.SignalQuality == SignalQuality.NoSignal)
    {
        UpdateUi(connected: true, signalOk: false);
        continue;
    }
    UpdateUi(connected: true, signalOk: true);
    ProcessData(data);
}

```

15. Advanced Patterns

WPF application with MVVM

```

public class EegViewModel : INotifyPropertyChanged, IAsyncDisposable
{
    private readonly NeuroSkySdk _sdk = new();
    private readonly CancellationTokenSource _cts = new();

    private int _attention, _meditation;
    private string _status = "Disconnected";

    public int Attention { get => _attention; private set { _attention = value;
OnPropertyChanged(); } }
    public int Meditation { get => _meditation; private set { _meditation = value;
OnPropertyChanged(); } }
    public string Status { get => _status; private set { _status = value;
OnPropertyChanged(); } }

    public async Task ConnectAsync(string macAddress)
    {
        _sdk.StateChanged += (_, state) =>
            App.Current.Dispatcher.Invoke(() => Status = state.ToString());

        await _sdk.ConnectAsync(macAddress);

        if (_sdk.State == ConnectionState.Error)
            return;

        await _sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);

        _ = Task.Run(async () =>
        {
            await foreach (var data in _sdk.DataStream(_cts.Token))
            {
                if (data.SignalQuality == SignalQuality.NoSignal) continue;

                App.Current.Dispatcher.Invoke(() =>
                {
                    Attention = data.Attention;
                    Meditation = data.Meditation;
                });
            }
        });
    }

    public event PropertyChangedEventHandler? PropertyChanged;
    private void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    public async ValueTask DisposeAsync()
    {
        _cts.Cancel();
        await _sdk.DisposeAsync();
    }
}

```

```

    }
}

```

Buffering 1 second of raw EEG for custom FFT

```

// Raw EEG arrives as 10 samples per packet.
// Buffer 512 samples to get exactly 1 second of data at 512 Hz.

var buffer = new List<int>(512);

await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.RawEeg.Count == 0) continue; // skip non-raw packets
    buffer.AddRange(data.RawEeg);

    if (buffer.Count >= 512)
    {
        var oneSecond = buffer.GetRange(0, 512).ToArray();
        buffer.RemoveRange(0, 512);

        var spectrum = MyFft.Compute(oneSecond, sampleRate: 512);
        DisplaySpectrum(spectrum);
    }
}

```

Recording session data to CSV

```

var filename = $"eeg_{DateTime.Now:yyyyMMdd_HH:mm:ss}.csv";

await using var writer = new StreamWriter(filename);
await writer.WriteLineAsync(
    "timestamp_ms,attention,meditation,poor_signal,signal_quality," +
    "delta,theta,low_alpha,high_alpha,low_beta,high_beta,low_gamma,mid_gamma");

await foreach (var d in sdk.DataStream(cts.Token))
{
    // Skip packets with no eSense data (raw EEG-only packets)
    if (d.Attention == 0 && d.Meditation == 0) continue;

    await writer.WriteLineAsync(
        $"{d.Timestamp},{d.Attention},{d.Meditation},{d.PoorSignal},{d.SignalQuality}," +
        $"{d.Delta},{d.Theta},{d.LowAlpha},{d.HighAlpha}," +
        $"{d.LowBeta},{d.HighBeta},{d.LowGamma},{d.MidGamma}");
}

Console.WriteLine($"Session saved to {filename}");

```

Producer / consumer separation with `Channel<T>`

If your processing is heavy and you don't want to block the data stream, decouple reading and processing with a bounded channel:

```
var channel = Channel.CreateBounded<BrainWaveData>(capacity: 100);

// Producer: reads from BLE and writes to channel
var producer = Task.Run(async () =>
{
    await foreach (var data in sdk.DataStream(cts.Token))
        await channel.Writer.WriteAsync(data, cts.Token);

    channel.Writer.Complete();
});

// Consumer: reads from channel and does heavy processing
var consumer = Task.Run(async () =>
{
    await foreach (var data in channel.Reader.ReadAllAsync(cts.Token))
        await HeavyProcessingAsync(data);
});

await Task.WhenAll(producer, consumer);
```

Packet timing & common pitfalls

In BLE mode, two characteristics transmit packets at different rates:

Characteristic	Fields	Rate
eSense 039afff8	Attention, Meditation, EEG bands	~1 Hz
RawEEG 039afff4	RawEeg (10 samples)	~51 Hz (512 Hz ÷ 10)

`ThinkGearParser` accumulates state across packets. Each `BrainWaveData` object contains the **latest accumulated value of every field**, regardless of which characteristic triggered the emit.

A common mistake is filtering on `Attention > 0` :

```
// Wrong - drops all packets in RawEEG-only sessions
await foreach (var data in sdk.DataStream(ct))
{
    if (data.Attention == 0) continue; // Attention is 0 when eSense is off
    // ...
}
```

If `StopESense` is sent (or eSense never starts), `Attention` stays at `0` permanently and this guard silently drops every packet.

Correct patterns:

```

// eSense session – filter by signal quality
await foreach (var data in sdk.DataStream(ct))
{
    if (data.SignalQuality == SignalQuality.NoSignal) continue;
    Console.WriteLine($"Attention: {data.Attention}");
}

// RawEEG-only session
await sdk.SendCommandAsync(NeuroSkyCommand.StopESense);
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);
await foreach (var data in sdk.DataStream(ct))
{
    if (data.RawEeg.Count > 0)
        foreach (var sample in data.RawEeg) ProcessRawSample(sample);
}

// eSense + RawEEG simultaneously – process only populated fields per packet
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg); // eSense already active
await foreach (var data in sdk.DataStream(ct))
{
    if (data.RawEeg.Count > 0) UpdateRawEegChart(data.RawEeg);
    if (data.Attention > 0)    UpdateEsenseUI(data);
}

```

16. Troubleshooting

Connection issues

Symptom	Likely cause	Solution
State becomes Error immediately on BLE	No BLE adapter, BLE disabled, or wrong MAC	Open Device Manager → Bluetooth, confirm adapter present and enabled; verify MAC
BLE connect succeeds but DataStream yields nothing	Handshake characteristic missing from device — v2.0.3+ now reports this as Error	Confirm device is a MindWave Mobile 2 (not a different headset advertising similar name)
BtClassic fails with access denied or not found	Device not paired in Windows	Pair via Settings → Bluetooth & other devices first
ConnectAsync never returns	Bluetooth adapter stuck	Disable / re-enable Bluetooth adapter in Device Manager
Connection drops after a few minutes idle	Windows Bluetooth power saving	Device Manager → Bluetooth adapter → Properties → Power Management → uncheck "Allow the computer to turn off this device"

Signal quality issues

Symptom	Likely cause	Solution
NoSignal immediately after connecting	Electrode not touching forehead	Adjust headset; press sensor firmly to skin
Poor signal persists after 30 seconds	Dry skin or dirty sensor	Wet sensor tip with water; clean forehead
Attention/meditation always 0	eSense needs warm-up	Wait 20–30 seconds after achieving Good signal
50 Hz / 60 Hz spike in raw EEG	No notch filter	Call <code>SendCommandAsync(NeuroSkyCommand.Notch60Hz)</code> after connecting
RawEeg always empty	<code>StartRawEeg</code> not called	Call <code>SendCommandAsync(NeuroSkyCommand.StartRawEeg)</code> after connecting

Build / runtime issues

Symptom	Likely cause	Solution
CS0246: 'NeuroSkySdk' not found	Missing using <code>NeuroSky.Sdk</code> ;	Add the using directive — a single one is enough (all types are flat)
CS0246: 'TransportMode' not found after v2.0.1	Using old <code>NeuroSky.Sdk.Transport.*</code> namespace	Remove using <code>NeuroSky.Sdk.Transport</code> ; — types are now in <code>NeuroSky.Sdk</code>
<code>PlatformNotSupportedException</code> at runtime	Wrong <code>TargetFramework</code>	Ensure <code>.csproj</code> has <code>net8.0-windows10.0.19041.0</code>
<code>FileNotFoundException: WinRT.Runtime.dll</code>	Package not restored	Run <code>dotnet restore</code>
Trimmed/AOT publish: BLE data never arrives	Old SDK (v2.0.1 / v2.0.2) shipped descriptor that didn't match flattened FQNs	Upgrade to v2.0.3+ — descriptor was corrected in v2.0.3

MSIX / packaged app issues

Symptom	Likely cause	Solution
BLE returns null device in packaged app	Missing capability	Add <code><DeviceCapability Name="bluetooth" /></code> to <code>Package.appxmanifest</code>
App rejected from Store for Bluetooth use	Missing capability declaration	Same — declare capability and re-submit

17. Testing

The SDK ships with an xUnit test suite for `ThinkGearParser` — the packet parser that runs identically on both BLE and BT Classic transports. These tests require no hardware or Bluetooth adapter.

Running the tests

```
dotnet test NeuroSky.Tests/NeuroSky.Tests.csproj
```

Or from the solution root:

```
dotnet test
```

Test runner output:

Test summary: total: 15, failed: 0, succeeded: 15, skipped: 0, duration: ~40 ms

What is covered (15 tests)

Test	Description
ParseESense_0xEA_ReturnsAttentionMeditationPoorSignal	Attention, Meditation, PoorSignal extraction
ParseESense_0xEA_TooShort_ReturnsNull	Short packet → returns null
ParseESense_0xEB_ReturnsDeltaThetaAlpha	Delta, Theta, LowAlpha, HighAlpha extraction
ParseESense_0xEC_ReturnsBetaGamma	LowBeta, HighBeta, LowGamma, MidGamma extraction
ParseRawEeg_Returns10Samples	20-byte raw EEG → 10 signed int samples
ParseRawEeg_SignedConversion_NegativeValue	Values > 32 768 converted to negative
ParseRawEeg_TooShort_ReturnsNull	Short packet → returns null
Parse_UnknownUuid_ReturnsNull	Unknown UUID → returns null
ParseByte_ValidPacket_ReturnsAttentionMeditation	BT Classic serial packet — Attention/Meditation
ParseByte_InvalidChecksum_ReturnsNull	Wrong checksum → returns null
ParseByte_PoorSignalCode_ReturnsPoorSignal	BT Classic PoorSignal (code 0x02)
SignalQuality_Thresholds(200, "NoSignal")	200 → NoSignal
SignalQuality_Thresholds(100, "Poor")	100 → Poor
SignalQuality_Thresholds(25, "Fair")	25 → Fair
SignalQuality_Thresholds(0, "Good")	0 → Good

Test location

```
NeuroSky.Tests/  
└── ThinkGearParserTests.cs
```

18. API Reference

NeuroSkySdk

Main entry point. Manages BLE/BT Classic transport selection and lifecycle.

```
public sealed class NeuroSkySdk : IDisposable
```

Member	Type	Description
State	ConnectionState	Current connection state (property, get-only)
StateChanged	event EventHandler<ConnectionState>	Fires whenever the state changes
ConnectAsync(string, TransportMode, CancellationToken)	Task	Initiate connection. Does NOT throw on connect failure — transitions to Error instead. Default mode: TransportMode.Ble. No automatic fallback.
FindDeviceAddressAsync(string, int, CancellationToken)	Task<string?>	Scan BLE advertisements; resolve device name → MAC. Default timeout 10 000 ms. Returns null on timeout.
DisconnectAsync()	Task	Gracefully disconnect the active transport
DataStream(CancellationToken)	IAsyncEnumerable<BrainWaveData>	Async stream of EEG packets; ends when connection drops or token cancels
SendCommandAsync(byte)	Task	Send a control byte to the headset
DisposeAsync()	ValueTask	Disconnect and release all Bluetooth resources

BrainWaveData

Immutable record emitted by `DataStream()`.

Property	Type	Range	Description
Timestamp	long	Unix ms (UTC)	Time this packet was received
PoorSignal	int	0~200	0 = perfect contact, 200 = no contact
Attention	int	0~100	eSense™ attention level (0 = not computed)
Meditation	int	0~100	eSense™ meditation level (0 = not computed)

Delta	int	0~∞	Delta band power, 0.5~2.75 Hz
Theta	int	0~∞	Theta band power, 3.5~6.75 Hz
LowAlpha	int	0~∞	Low Alpha, 7.5~9.25 Hz
HighAlpha	int	0~∞	High Alpha, 10~11.75 Hz
LowBeta	int	0~∞	Low Beta, 13~16.75 Hz
HighBeta	int	0~∞	High Beta, 18~29.75 Hz
LowGamma	int	0~∞	Low Gamma, 31~39.75 Hz
MidGamma	int	0~∞	Mid Gamma, 41~49.75 Hz
RawEeg	IReadOnlyList<int>	-32768~32767	512 Hz ADC samples (10 per packet)
EyeBlink	int	0~255	Eye blink intensity; 0 = no blink
SignalQuality	SignalQuality	enum	Derived from PoorSignal

TransportMode

Controls which Bluetooth protocol `ConnectAsync` uses.

```
public enum TransportMode { Ble, BtClassic }
```

Value	Behavior	Pairing required?
Ble	BLE GATT only (default)	No
BtClassic	RFCOMM SPP only	Yes

ConnectionState

```
public enum ConnectionState { Disconnected, Scanning, Connecting, Connected, Error }
```

Value	Meaning
Disconnected	No active connection
Scanning	BLE only — resolving target device
Connecting	Establishing GATT/RFCOMM connection
Connected	Data stream active
Error	Connection attempt failed (device not found, GATT discovery failed, handshake characteristic missing, RFCOMM unavailable)

SignalQuality

Derived from `BrainWaveData.PoorSignal` .

Value	PoorSignal	Reliability	Recommended action
Good	0	Excellent	Use all data
Fair	1~50	Acceptable	Use data; minor noise present
Poor	51~199	Unreliable	Prompt user to adjust headset
NoSignal	200	No data	Prompt user to put on headset

ITransport (interface)

Common interface implemented by `BleTransport` , `BtClassicTransport` , and `SimulatorTransport` .

```
public interface ITransport : IDisposable
{
    ConnectionState State { get; }
    event EventHandler<ConnectionState> StateChanged;

    IEnumerable<BrainWaveData> DataStream(CancellationToken ct = default);
    Task ConnectAsync(string deviceAddress, CancellationToken ct = default);
    Task DisconnectAsync();
    Task SendCommandAsync(byte cmd);
}
```

SimulatorTransport

```
public sealed class SimulatorTransport : ITransport
{
    public enum Mode { Random, Focused, Relaxed, PoorSignal }
    public void SetMode(Mode mode);
}
```

Mode	Description
Random	Random Attention/Meditation each tick
Focused	Attention 70100, Meditation 4060
Relaxed	Attention 2050, Meditation 70100
PoorSignal	PoorSignal 150~200, Attention 0, Meditation 0

NeuroSkyCommand

```
public static class NeuroSkyCommand
```

Constant	Byte	When to use
Notch60Hz	0x1C	Power grid is 60 Hz (Korea, USA)
Notch50Hz	0x1B	Power grid is 50 Hz (Europe, China)
StartRawEeg	0x15	Enable raw EEG waveform (disabled by default)
StopRawEeg	0x16	Disable raw EEG waveform
StartESense	0x17	Enable Attention/Meditation (auto-sent on BLE connect)
StopESense	0x18	Disable Attention/Meditation

NeuroSkyUuid

BLE GATT UUID constants — for advanced users who need to identify characteristics or extend the parser.

```
public static class NeuroSkyUuid
```

Constant	UUID	Purpose
ESense	039afff8----	eSense (Attention/Meditation/bands) notify
Handshake	039affa0----	Handshake / command write
RawEeg	039afff4----	Raw EEG notify
Spp	00001101----	BT Classic RFCOMM SPP
Manufacturer, ModelNumber, SerialNumber, HwRevision, FwRevision, SwRevision	standard BLE	Device Information Service